ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмної інженерії»

Звіт
з лабораторної роботи №1
з дисципліни
«Основи мови Dart»

Виконала:
ст. гр. ПЗПІ-23-3
Давиденко П.І.

05.11.2026

1 МЕТА РОБОТИ ТА ЗАВДАННЯ ЗА ВАРІАНТОМ

1.1 Мета роботи

Отримання здобувачами навичок оголошення функцій із різними видами параметрів, побудови конвеєрів обробки колекцій засобами Iterable, а також обробки рядків, дат і регулярних виразів мовою Dart.

1.2 Варіант

Варіант 4 - «Розклад занять групи».

1.3 Репозиторій

https://github.com/NureDavydenkoPolina/Dart/lab1

2 ОСНОВНА ЧАСТИНА

2.1 Створення проєкту (пункт 1)

Проєкт створено командою dart create -t console lab1. Назва пакета написана в стилі lower_snake_case, як вимагає методичка. У pubspec.yaml є одна залежність - lints у розділі dev_dependencies. Звичайних залежностей немає.

У analysis_options.yaml підключено набір package:lints/recommended.yaml. Я додала ще два правила, які лістинг 1 показує повністю. Файл .gitignore містить .dart_tool/ і build/.

Лістинг 1 – Файл analysis_options.yaml

```
include: package:lints/recommended.yaml

linter:
  rules:
    - prefer_single_quotes
    - prefer_final_locals
```

Код розділено на дві частини. У bin/lab1.dart лежить тільки main, який викликає функції й друкує результат. Уся логіка та дані лежать у lib/. Так логіку можна використати в наступних лабораторних і перевіряти тестами [8].

2.2 Вхідні дані (пункт 2)

Дані оголошено константою rawSchedule у файлі lib/lab1_data.dart. Це 19 рядків формату дисципліна;викладач;тип;день;початок;аудиторія;дата запису. Перші 14 рядків правильні, останні 5 - пошкоджені навмисно. Методичка вимагає три, але я зробила п’ять, щоб перевірити і розбір, і валідатор. Види помилок зібрано в таблиці 1.

Таблиця 1 – Навмисно пошкоджені рядки

Рядок	Що не так	Де виявляється
15	бракує поля (6 замість 7)	parseLine
16	час 1x:30 не є числом	parseLine
17	дата 2026-13-45, тринадцятого місяця немає	parseLine
18	час 07:00 раніше за 08:00	validateLesson
19	тип «семінар» немає в переліку	validateLesson

2.3 Розбір рядка (пункт 3)

Функція parseLine розбиває рядок за символом ; і обрізає пробіли. Потім вона перевіряє кількість полів, розбирає час і дату. Для цього я використала DateTime.tryParse та int.tryParse, а не parse у блоці try/catch. Пошкоджений рядок у вхідних даних - це очікувана ситуація, а не виняток [3].

Є одна особливість. DateTime.tryParse не відхиляє неіснуючі місяць чи день, а переносить надлишок на наступний період. Я помітила це під час запуску: дата 2026-13-45 перетворилась на 14 лютого 2027 і пройшла розбір. Тому після розбору я порівнюю відформатовану дату з вихідним рядком (formatDate(addedOn) != dateText). Якщо вони різні, рядок вважається пошкодженим.

Функція повертає запис ParseResult із двох полів: заняття або текст помилки. Одне з них завжди null. Через це програма не падає на жодному з пошкоджених рядків. Час розбирає окрема функція parseTime. Вона перевіряє формат регулярним виразом, а потім перетворює години й хвилини в int.

Лістинг 2 – Функція parseLine (скорочено)

```
ParseResult parseLine(String line) {
  final fields = line.split(fieldSeparator).map((f) => f.trim()).toList();
  if (fields.length != fieldCount) {
    return (
      lesson: null,
      error: 'очікувалось $fieldCount полів, знайдено ${fields.length}',
    );
  }
  final [subject, teacher, type, day, timeText, room, dateText] = fields;
  final startMinutes = parseTime(timeText);
  if (startMinutes == null) {
    return (lesson: null, error: 'час «$timeText» не схожий на ГГ:ХХ');
  }
  final addedOn = DateTime.tryParse(dateText);
  if (addedOn == null || formatDate(addedOn) != dateText) {
    return (lesson: null, error: 'некоректна дата «$dateText»');
  }
  // ... далі збирається запис Lesson
}
```

2.4 Валідатор (пункт 4)

Функція validateLesson повертає список порушень. Порожній список означає, що заняття правильне. Перевіряються день тижня, тип заняття, діапазон часу (від

08:00 до 19:00) і те, що дата запису не з майбутнього. Поточну дату я передаю параметром now. Тому функція не залежить від годинника, і її легко перевіряти.

Методичка згадує рік, кількість і оцінку. У варіанті 4 таких полів немає, тому діапазони перевіряються для часу й дати. Функція loadLessons об'єднує розбір і валідацію. Правильні заняття йдуть далі, а опис кожної проблеми з номером рядка потрапляє в окремий список.

Лістинг 3 – Функція validateLesson (скорочено)

```
List<String> validateLesson(Lesson lesson, {required DateTime now}) {
  final start = lesson.startMinutes;
  return [
    if (!weekdays.contains(lesson.day)) 'невідомий день «${lesson.day}»',
    // ... так само перевіряється тип заняття
    if (start < earliestStartMinutes || start > latestStartMinutes)
      'початок ${formatTime(start)} поза межами ...',
    if (lesson.addedOn.isAfter(now))
      'дата запису ${formatDate(lesson.addedOn)} у майбутньому',
  ];
}
```

2.5 Конвеєри обробки (пункт 5)

У програмі є три конвеєри (лістинг 4). Перший фільтрує заняття за умовою (усі заняття в понеділок). Другий перетворює заняття на рядки для виводу. Третій згортає колекцію до числа: сумарна тривалість у хвилинах. Крім того, startRange шукає найраніший і найпізніший початок через reduce(min) та reduce(max). Тут reduce безпечний, бо спочатку перевіряється, що колекція не порожня. countByType рахує заняття кожного типу через fold. Цикли з накопичувальною змінною я не використовувала [4].

Лістинг 4 – Три конвеєри

```
// 1. Фільтрація
final monday = lessons.where(byDay('понеділок')).toList();

// 2. Перетворення
final lectures = lessons.where(byType('лекція')).map(describeLesson);

// 3. Згортання
int totalMinutes(Iterable<Lesson> lessons) =>
    lessons.map(durationOf).fold(0, (sum, value) => sum + value);
```

2.6 Групування (пункт 6)

Функція groupByDay збирає заняття в Map<String, List<Lesson>>. Ключ - день тижня, значення - список занять цього дня. Для цього використано fold з порожньою мапою. Потім statsByDay для кожної групи рахує кількість занять і сумарну тривалість. Результат - запис DayStats.

Лістинг 5 – Групування за днем тижня

```
Map<String, List<Lesson>> groupByDay(Iterable<Lesson> lessons) {
  return lessons.fold<Map<String, List<Lesson>>>({}, (groups, lesson) {
    groups.putIfAbsent(lesson.day, () => <Lesson>[]).add(lesson);
    return groups;
  });
}
```

2.7 Сортування (пункт 7)

Є два критерії сортування. sortBySchedule сортує за днем тижня, а потім за часом. Для дня береться індекс у списку weekdays, бо за алфавітом дні йшли б неправильно. sortBySubject сортує за назвою дисципліни, а потім за часом. Напрям змінює іменований параметр ascending: якщо він false, результат порівняння міняє знак. Спочатку функція робить копію списку через toList(), тому вхідний список не змінюється.

Лістинг 6 – Сортування за розкладом

```
List<Lesson> sortBySchedule(
  Iterable<Lesson> lessons, {
  bool ascending = true,
}) {
  final sorted = lessons.toList();
  sorted.sort((a, b) {
    final dayOrder =
        weekdays.indexOf(a.day).compareTo(weekdays.indexOf(b.day));
    final result = dayOrder != 0
        ? dayOrder
        : a.startMinutes.compareTo(b.startMinutes);
    return ascending ? result : -result;
  });
  return sorted;
}
```

2.8 Звіт (пункт 8)

Функція buildReport має один обов’язковий параметр і три іменовані зі значеннями за замовчуванням: topCount = 5, ascending = true, separator = ' | '. Текст

збирається в StringBuffer, а не через +=. Рядки в такому разі не копіюються при кожному додаванні. У програмі звіт викликається двічі: зі значеннями за замовчуванням і зі зміненими параметрами (зворотний порядок, інший роздільник).

Лістинг 7 – Функція buildReport (скорочено)

```
String buildReport(
  List<Lesson> lessons, {
  int topCount = 5,
  bool ascending = true,
  String separator = ' | ',
}) {
  final nextNumber = makeCounter();
  final top = sortBySchedule(lessons, ascending: ascending).take(topCount);
  // ...
  final buffer = StringBuffer()
    ..writeln('Усього занять: ${lessons.length}')
    ..writeln('Перші $topCount за розкладом:')
    ..writeAll(top.map(/* рядок звіту */), '\n');
  // ...
  return buffer.toString();
}
```

2.9 Замикання (пункт 9)

У роботі є три замикання. byDay(day) повертає функцію-предикат, яка порівнює день заняття з параметром day. Замикання захоплює саме цей параметр. Він живе, поки жива повернена функція. byType працює так само.

makeCounter() повертає лічильник. Він захоплює локальну змінну count. Кожен виклик збільшує її на одиницю. Після виходу з makeCounter змінна не зникає, бо на неї посилається повернена функція. Кожен новий виклик makeCounter() створює свій окремий count. У buildReport лічильник нумерує рядки звіту.

Лістинг 8 – Замикання

```
bool Function(Lesson) byDay(String day) => (lesson) => lesson.day == day;

int Function() makeCounter() {
  var count = 0;
  return () => ++count;
}
```

2.10 Регулярні вирази (пункт 10)

Для перевірки формату часу є timePattern з іменованими групами hour і minute. Через них код читається краще, ніж з group(1). Для маскування є teacherPattern з

групами first і rest. Метод replaceAllMapped лишає першу літеру прізвища, а решту букв замінює зірочками. Наприклад, «Баличев М.Д.» стане «Б****** М.Д.». Вираз прив’язаний до початку рядка, тому ініціали не змінюються. Колбек replaceAllMapped отримує тип Match, а namedGroup є лише в RegExpMatch. Тому перед викликом стоїть перевірка match is! RegExpMatch. Прапорець unicode: true потрібен, щоб працював клас p{L} для української. Обидва шаблони записані сирими рядками r'…', щоб не подвоювати зворотні скісні риски [3].

Лістинг 9 – Регулярні вирази

```
final RegExp timePattern = RegExp(r'^(?<hour>\d{1,2}):(?<minute>\d{2})$');

final RegExp teacherPattern = RegExp(
  r'^(?<first>\p{L})(?<rest>\p{L}+)',
  unicode: true,
);

String maskTeacher(String teacher) {
  return teacher.replaceAllMapped(teacherPattern, (match) {
    if (match is! RegExpMatch) return match[0] ?? '';
    final first = match.namedGroup('first') ?? '';
    final rest = match.namedGroup('rest') ?? '';
    return '$first${'*' * rest.length}';
  });
}
```

2.11 Арифметика дат (пункт 11)

Вік запису в днях рахує ageInDays. Вона віднімає дату запису від поточної через DateTime.difference. Результат - об’єкт Duration, з якого береться inDays. Поточна дата передається параметром, як і у валідаторі. Дату з рядка програма отримує через DateTime.tryParse.

Лістинг 10 – Вік запису в днях

```
int ageInDays(Lesson lesson, {required DateTime now}) {
  final Duration age = now.difference(lesson.addedOn);
  return age.inDays;
}
```

3 РЕЗУЛЬТАТИ ВИКОНАННЯ



Рисунок 1 – Результат команди dart analyze

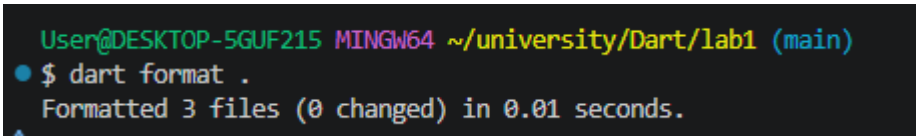


Рисунок 2 – Результат команди dart format .

Вивід програми:

```
$ dart run
Building package executable...
Built lab1:lab1.

=== Проблемні рядки ===
рядок 15: очікувалось 7 полів, знайдено 6
рядок 16: час «1x:30» не схожий на ГГ:ХХ
рядок 17: некоректна дата «2026-13-45»
рядок 18: початок 07:00 поза межами 08:00–19:00
рядок 19: невідомий тип «семінар»
Коректних занять: 14 з 19

=== Конвеєр 1: фільтрація (понеділок) ===
понеділок | 08:30 | Програмування мовою Dart | лекція | Ж-301
понеділок | 10:10 | Бази даних | лекція | Ж-301
понеділок | 13:30 | Операційні системи | лабораторна | Л-204

=== Конвеєр 2: перетворення (лекції як рядки) ===
понеділок | 08:30 | Програмування мовою Dart | лекція | Ж-301
вівторок | 10:10 | Якість ПЗ та тестування | лекція | Ж-205
понеділок | 10:10 | Бази даних | лекція | Ж-301
середа | 08:30 | Алгоритми та структури даних | лекція | Ж-205
п’ятниця | 10:10 | Дискретна математика | лекція | Ж-301
четвер | 13:30 | Операційні системи | лекція | Ж-205

=== Конвеєр 3: згортання ===
Годин лекцій на тиждень: 9.0
Годин усього: 21.0
Початок занять: від 08:30 до 13:30
За типами: {лекція: 6, лабораторна: 3, практика: 5}

=== Сортування за дисципліною (за спаданням) ===
четвер | 11:50 | Якість ПЗ та тестування | лабораторна | Л-210
вівторок | 10:10 | Якість ПЗ та тестування | лекція | Ж-205
середа | 13:30 | Фізична культура | практика | Спортзал
```

```
=== Звіт ===
Усього занять: 14
Перші 4 за розкладом:
1. понеділок | 08:30 | Програмування мовою Dart | лекція | Ж-301
2. понеділок | 10:10 | Бази даних | лекція | Ж-301
3. понеділок | 13:30 | Операційні системи | лабораторна | Л-204
4. вівторок | 10:10 | Якість ПЗ та тестування | лекція | Ж-205
Навантаження за днями:
понеділок: 3 зан., 4.5 год
вівторок: 3 зан., 4.5 год
середа: 3 зан., 4.5 год
четвер: 3 зан., 4.5 год
п’ятниця: 2 зан., 3.0 год

Звіт у зворотному порядку:
Усього занять: 14
Перші 3 за розкладом:
1. п’ятниця · 10:10 · Дискретна математика · лекція · Ж-301
2. п’ятниця · 08:30 · Англійська мова · практика · Г-305
3. четвер · 13:30 · Операційні системи · лекція · Ж-205
Навантаження за днями:
понеділок: 3 зан., 4.5 год
вівторок: 3 зан., 4.5 год
середа: 3 зан., 4.5 год
четвер: 3 зан., 4.5 год
п’ятниця: 2 зан., 3.0 год

=== Регулярний вираз: маскування викладачів ===
Б****** М.Д.
Б****** М.Д.
К******** О.П.

=== Вік записів у днях ===
Програмування мовою Dart: 65 дн.
Програмування мовою Dart: 65 дн.
Якість ПЗ та тестування: 65 дн.

=== Пошук за аудиторією ===
середа | 10:10 | Програмування мовою Dart | лабораторна | Л-204
```

4 ВИСНОВКИ

У цій лабораторній роботі написано консольну програму для розбору, перевірки та підрахунку розкладу занять групи. Програма не використовує класів. Дані зберігаються в записах і колекціях, а обробка виконується функціями.

Навчилась оголошувати функції з іменованими параметрами та значеннями за замовчуванням, повертати з функції запис із двох значень, будувати конвеєри where, map, fold замість циклів. Також опанувала групування в Map, сортування з перемикачем напряму, збирання тексту через StringBuffer, замикання, регулярні вирази з іменованими групами та арифметику дат.

5 ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Effective Dart: style, documentation, usage, design [Електронний ресурс] – Режим доступу: https://dart.dev/effective-dart – Загол. з екрану.

2. Functions [Електронний ресурс] – Режим доступу: https://dart.dev/language/functions – Загол. з екрану.

3. Collections [Електронний ресурс] – Режим доступу: https://dart.dev/language/collections – Загол. з екрану.

4. Iterable collections [Електронний ресурс] – Режим доступу: https://dart.dev/libraries/collections/iterables – Загол. з екрану.

5. Records [Електронний ресурс] – Режим доступу: https://dart.dev/language/records – Загол. з екрану.

6. Patterns [Електронний ресурс] – Режим доступу: https://dart.dev/language/patterns – Загол. з екрану.

7. Dart API reference [Електронний ресурс] – Режим доступу: https://api.dart.dev – Загол. з екрану.

8. Package layout conventions [Електронний ресурс] – Режим доступу: https://dart.dev/tools/pub/package-layout – Загол. з екрану.

9. Customizing static analysis [Електронний ресурс] – Режим доступу: https://dart.dev/tools/analysis – Загол. з екрану.

10. Linter rules [Електронний ресурс] – Режим доступу: https://dart.dev/tools/linter-rules – Загол. з екрану.